

Portal & Mobile Live Coding Master

Candidate: Irshath Ahamed | Senior Flutter Developer (Portal/Mobile)

Senior Flutter Developer — Live Coding Master: Mobile & Portal Focus

This handbook contains **7 production-grade live coding challenges** frequently asked in Senior/Lead Flutter interviews for **Portal, Tablet & Mobile** roles. Every challenge includes the **Problem Statement, Edge Cases to Discuss with Interviewers, and Complete Production Code**.

Index of Live Coding Challenges

1. [Challenge 1: Adaptive Master-Detail Layout with Synchronized State](#)
2. [Challenge 2: High-Performance Virtualized Data Table with Sorting & Filtering](#)
3. [Challenge 3: Unified Adaptive Modal \(Bottom Sheet on Mobile vs Constrained Dialog on Portal\)](#)
4. [Challenge 4: Responsive Form with Multi-Column Breakpoints & Keyboard Shortcuts](#)
5. [Challenge 5: Cross-Platform Image Picker & Uploader \(Web Blob vs Mobile Native File\)](#)
6. [Challenge 6: Timesheet Shift Calculator with 24-Hour Auto-Capping & Overtime](#)
7. [Challenge 7: Reactive Offline-First Cache-Aside Repository \(Stream + In-Memory L1 Cache\)](#)

Challenge 1: Adaptive Master-Detail Layout with Synchronized State

The Interviewer's Prompt:

"Build an adaptive Master-Detail view for an Employee Directory. On mobile (<768px), tapping an employee pushes a new screen. On tablet/portal (≥768px), display a two-pane layout with the list on the left and the detail pane on the right. If the user resizes their browser window from tablet to mobile, state must not be lost."

Key Senior Points to Clarify:

- "Should the right-hand pane have a placeholder state when no employee is selected?" (Yes).
- "We should use `LayoutBuilder` and `MediaQuery.sizeOf(context)` rather than `MediaQuery.of(context).size` to avoid rebuilding on keyboard or inset changes."

Complete Production Code:

```
import 'package:flutter/material.dart';

class Employee {
  final String id;
  final String name;
  final String role;
  final String department;
  final double totalHours;

  const Employee({
    required this.id,
    required this.name,
    required this.role,
    required this.department,
    required this.totalHours,
  });
}

class AdaptiveMasterDetailScreen extends StatefulWidget {
  final List<Employee> employees;

  const AdaptiveMasterDetailScreen({super.key, required this.employees});

  @override
  State<AdaptiveMasterDetailScreen> createState() => _AdaptiveMasterDetailScreenState();
}

class _AdaptiveMasterDetailScreenState extends State<AdaptiveMasterDetailScreen> {
  Employee? _selectedEmployee;

  @override
  void initState() {
    super.initState();
    // Default select first item on wide screens if available
    if (widget.employees.isNotEmpty) {
      _selectedEmployee = widget.employees.first;
    }
  }

  void _onEmployeeTapped(Employee emp, bool isWideScreen) {
    setState(() => _selectedEmployee = emp);

    if (!isWideScreen) {
      // Mobile Flow: Navigate to standalone detail page
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) => Scaffold(
            appBar: AppBar(title: Text(emp.name)),
            body: EmployeeDetailPane(employee: emp),
          ),
        ),
      );
    }
  }

  @override
```

```

Widget build(BuildContext context) {
  final screenWidth = MediaQuery.sizeOf(context).width;
  final bool isWideScreen = screenWidth >= 768;

  final listPane = ListView.separated(
    itemCount: widget.employees.length,
    separatorBuilder: (_, __) => const Divider(height: 1),
    itemBuilder: (context, index) {
      final emp = widget.employees[index];
      final isSelected = isWideScreen && _selectedEmployee?.id == emp.id;

      return MouseRegion(
        cursor: SystemMouseCursors.click,
        child: ListTile(
          selected: isSelected,
          selectedTileColor: Theme.of(context).colorScheme.primaryContainer.withValues(alpha: 0.5),
          leading: CircleAvatar(child: Text(emp.name[0])),
          title: Text(emp.name, style: const TextStyle(fontWeight: FontWeight.w600)),
          subtitle: Text('${emp.role} • ${emp.department}'),
          trailing: Chip(
            label: Text('${emp.totalHours.toStringAsFixed(1)} hrs'),
            backgroundColor: Colors.blue.shade50,
          ),
          onTap: () => _onEmployeeTapped(emp, isWideScreen),
        ),
      );
    },
  );

  return Scaffold(
    appBar: AppBar(
      title: const Text('Workforce Directory'),
      elevation: 0.5,
    ),
    body: LayoutBuilder(
      builder: (context, constraints) {
        if (!isWideScreen) {
          // Mobile: Single list view
          return listPane;
        }

        // Tablet / Portal: Side-by-Side Master-Detail Split
        return Row(
          children: [
            SizedBox(
              width: constraints.maxWidth > 1200 ? 380 : 320,
              child: listPane,
            ),
            const VerticalDivider(width: 1),
            Expanded(
              child: _selectedEmployee != null
                ? EmployeeDetailPane(employee: _selectedEmployee!)
                : const Center(
                    child: Text(
                      'Select an employee from the directory to inspect timesheets',
                      style: TextStyle(color: Colors.grey, fontSize: 16),
                    ),
                  ),
            ),
          ],
        );
      },
    ),
  );
}

```

```

    );
  },
),
);
}
}

class EmployeeDetailPane extends StatelessWidget {
  final Employee employee;

  const EmployeeDetailPane({super.key, required this.employee});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              CircleAvatar(radius: 36, child: Text(employee.name[0], style: const TextStyle(fontS
              const SizedBox(width: 16),
              Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(employee.name, style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                  Text('${employee.role} | ${employee.department}', style: TextStyle(color: Color
                ],
              ),
            ],
          ),
          const SizedBox(height: 24),
          Card(
            elevation: 0,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
              side: BorderSide(color: Colors.grey.shade200),
            ),
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceAround,
                children: [
                  _MetricBadge(label: 'Total Hours', value: '${employee.totalHours.toStringAsFixe
                  _MetricBadge(label: 'Regular Hours', value: '8.0 h'),
                  _MetricBadge(label: 'Overtime', value: '2.5 h'),
                  _MetricBadge(label: 'Status', value: 'Active'),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}

```

```

class _MetricBadge extends StatelessWidget {
  final String label;

```

```
final String value;
const _MetricBadge({required this.label, required this.value});

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text(label, style: const TextStyle(fontSize: 12, color: Colors.grey)),
      const SizedBox(height: 4),
      Text(value, style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
    ],
  );
}
```

Challenge 2: High-Performance Virtualized Data Table with Sorting & Filtering

The Interviewer's Prompt:

"On the web portal, administrators need to review 5,000 timesheet records. Standard `DataTable` rebuilds all rows and lags the browser. Implement a fast, virtualized data table with real-time text filtering and column sorting (by Name, Date, Hours)."

Key Senior Points to Clarify:

- "Instead of wrapping a raw `DataTable` inside a `SingleChildScrollView` (which causes DOM choking on web), we use a custom virtualized `ListView.builder` with a sticky header row."*

Complete Production Code:

```
import 'package:flutter/material.dart';

class TimesheetRowData {
  final String id;
  final String employeeName;
  final DateTime date;
  final double regularHours;
  final double overtimeHours;

  double get totalHours => regularHours + overtimeHours;

  const TimesheetRowData({
    required this.id,
    required this.employeeName,
    required this.date,
    required this.regularHours,
    required this.overtimeHours,
  });
}

enum SortColumn { name, date, hours }

class VirtualizedPortalDataTable extends StatefulWidget {
  final List<TimesheetRowData> records;

  const VirtualizedPortalDataTable({super.key, required this.records});

  @override
  State<VirtualizedPortalDataTable> createState() => _VirtualizedPortalDataTableState();
}

class _VirtualizedPortalDataTableState extends State<VirtualizedPortalDataTable> {
  String _searchQuery = '';
  SortColumn _sortColumn = SortColumn.date;
  bool _isAscending = false;

  List<TimesheetRowData> get _filteredAndSortedRecords {
    var list = widget.records.where((r) {
      if (_searchQuery.isEmpty) return true;
      return r.employeeName.toLowerCase().contains(_searchQuery.toLowerCase());
    }).toList();

    list.sort((a, b) {
      int cmp;
      switch (_sortColumn) {
        case SortColumn.name:
          cmp = a.employeeName.toLowerCase().compareTo(b.employeeName.toLowerCase());
          break;
        case SortColumn.date:
          cmp = a.date.compareTo(b.date);
          break;
        case SortColumn.hours:
          cmp = a.totalHours.compareTo(b.totalHours);
          break;
      }
      return _isAscending ? cmp : -cmp;
    });
  }
}
```

```

    return list;
}

void _onSort(SortColumn column) {
  setState(() {
    if (_sortColumn == column) {
      _isAscending = !_isAscending;
    } else {
      _sortColumn = column;
      _isAscending = true;
    }
  });
}

@override
Widget build(BuildContext context) {
  final displayedRecords = _filteredAndSortedRecords;

  return Scaffold(
    appBar: AppBar(title: const Text('Timesheet Audit Table (Web Portal)'),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          // Search Input Bar
          TextField(
            decoration: InputDecoration(
              prefixIcon: const Icon(Icons.search),
              hintText: 'Filter by employee name...',
              border: OutlineInputBorder(borderRadius: BorderRadius.circular(8)),
              contentPadding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
            ),
            onChanged: (val) => setState(() => _searchQuery = val),
          ),
          const SizedBox(height: 16),

          // Sticky Header Row
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
            decoration: BoxDecoration(
              color: Colors.blueGrey.shade50,
              borderRadius: const BorderRadius.vertical(top: Radius.circular(8)),
            ),
            child: Row(
              children: [
                Expanded(
                  flex: 3,
                  child: _buildHeaderCell('Employee Name', SortColumn.name),
                ),
                Expanded(
                  flex: 2,
                  child: _buildHeaderCell('Date', SortColumn.date),
                ),
                Expanded(
                  flex: 2,
                  child: _buildHeaderCell('Regular', null),
                ),
                Expanded(
                  flex: 2,

```

```

        child: _buildHeaderCell('Overtime', null),
      ),
      Expanded(
        flex: 2,
        child: _buildHeaderCell('Total Hours', SortColumn.hours),
      ),
    ],
  ),
),
// Virtualized Body Rows (Only visible rows are mounted in memory)
Expanded(
  child: displayedRecords.isEmpty
    ? const Center(child: Text('No records match your filter'))
    : ListView.builder(
        itemCount: displayedRecords.length,
        itemBuilder: (context, index) {
          final row = displayedRecords[index];
          final isAlt = index.isOdd;

          return Container(
            color: isAlt ? Colors.grey.shade50 : Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
            child: Row(
              children: [
                Expanded(
                  flex: 3,
                  child: Text(row.employeeName, style: const TextStyle(fontWeight:
                ),
                Expanded(
                  flex: 2,
                  child: Text('${row.date.year}-${row.date.month.toString().padLeft
                ),
                Expanded(
                  flex: 2,
                  child: Text('${row.regularHours.toStringAsFixed(1)} h'),
                ),
                Expanded(
                  flex: 2,
                  child: Text(
                    '${row.overtimeHours.toStringAsFixed(1)} h',
                    style: TextStyle(
                      color: row.overtimeHours > 0 ? Colors.orange.shade800 : Color
                      fontWeight: row.overtimeHours > 0 ? FontWeight.bold : FontWei
                    ),
                  ),
                ),
                Expanded(
                  flex: 2,
                  child: Text(
                    '${row.totalHours.toStringAsFixed(1)} h',
                    style: const TextStyle(fontWeight: FontWeight.bold),
                  ),
                ),
              ],
            ),
          );
        },
      ),
),

```

```

        ],
      ),
    ),
  );
}

Widget _buildHeaderCell(String label, SortColumn? column) {
  final isCurrent = _sortColumn == column;

  return MouseRegion(
    cursor: column != null ? SystemMouseCursors.click : SystemMouseCursors.basic,
    child: GestureDetector(
      onTap: column != null ? () => _onSort(column) : null,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(label, style: const TextStyle(fontWeight: FontWeight.bold)),
          if (column != null && isCurrent) ...[
            SizedBox(width: 4),
            Icon(
              _isAscending ? Icons.arrow_upward : Icons.arrow_downward,
              size: 14,
              color: Colors.blue.shade700,
            ),
          ],
        ],
      ),
    ),
  );
}

```

Challenge 3: Unified Adaptive Modal (Bottom Sheet on Mobile vs Constrained Dialog on Portal)

The Interviewer's Prompt:

"Write a reusable modal manager function: `showAdaptiveModal()` . On mobile, it must present a draggable bottom sheet. On tablet and web portal, it must display a centered modal dialog constrained to a maximum width of 550px."

Complete Production Code:

```
import 'package:flutter/material.dart';

Future<T?> showAdaptiveModal<T>({
  required BuildContext context,
  required String title,
  required Widget Function(BuildContext context) builder,
  double maxWidth = 550.0,
}) {
  final isMobile = MediaQuery.sizeOf(context).width < 768;

  if (isMobile) {
    // Mobile: Native Bottom Sheet with drag handle
    return showModalBottomSheet<T>({
      context: context,
      isScrollControlled: true,
      shape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(16)),
      ),
      builder: (ctx) => Padding(
        padding: EdgeInsets.only(bottom: MediaQuery.of(ctx).viewInsets.bottom),
        child: SafeArea(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const SizedBox(height: 8),
              Container(
                width: 40,
                height: 4,
                decoration: BoxDecoration(
                  color: Colors.grey.shade400,
                  borderRadius: BorderRadius.circular(2),
                ),
              ),
            ],
          ),
          Padding(
            padding: const EdgeInsets.all(16.0),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Text(title, style: Theme.of(ctx).textTheme.titleMedium?.copyWith(fontWeight:
                  IconButton(icon: const Icon(Icons.close), onPressed: () => Navigator.pop(ctx)
                ],
              ),
            ),
            const Divider(height: 1),
            Flexible(child: builder(ctx)),
          ],
        ),
      ),
    );
  }

  // Tablet & Desktop Portal: Centered Constrained Modal
  return showDialog<T>({
    context: context,
    builder: (ctx) => Center(
      child: ConstrainedBox(
```

```

constraints: BoxConstraints(maxWidth: maxWidth),
child: Dialog(
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text(title, style: Theme.of(ctx).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.normal)),
            IconButton(icon: const Icon(Icons.close), onPressed: () => Navigator.pop(ctx))
          ],
        ),
      ),
      const Divider(height: 1),
      Padding(
        padding: const EdgeInsets.all(20.0),
        child: builder(ctx),
      ),
    ],
  ),
),
);
}

```

Challenge 4: Responsive Form with Multi-Column Breakpoints & Keyboard Shortcuts

The Interviewer's Prompt:

"Create a user creation form that renders as 1 column on mobile (<600px), 2 columns on tablet (600px–1000px), and 3 columns on wide portal screens (>1000px). On web/desktop, support keyboard shortcuts: `Ctrl/Cmd + S` to save and `Esc` to cancel."

Complete Production Code:

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

// 1. Define Intent classes for Shortcuts
class SaveIntent extends Intent {
  const SaveIntent();
}

class CancelIntent extends Intent {
  const CancelIntent();
}

class ResponsiveKeyboardForm extends StatefulWidget {
  const ResponsiveKeyboardForm({super.key});

  @override
  State<ResponsiveKeyboardForm> createState() => _ResponsiveKeyboardFormState();
}

class _ResponsiveKeyboardFormState extends State<ResponsiveKeyboardForm> {
  final _formKey = GlobalKey<FormState>();

  void _onSave() {
    if (_formKey.currentState?.validate() ?? false) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Form Submitted Successfully! (Shortcuts supported)'), backg
      );
    }
  }

  void _onCancel() {
    Navigator.of(context).maybePop();
  }

  @override
  Widget build(BuildContext context) {
    return Shortcuts(
      shortcuts: <LogicalKeySet, Intent>{
        LogicalKeySet(LogicalKeyboardKey.meta, LogicalKeyboardKey.keyS): const SaveIntent(),
        LogicalKeySet(LogicalKeyboardKey.control, LogicalKeyboardKey.keyS): const SaveIntent(),
        LogicalKeySet(LogicalKeyboardKey.escape): const CancelIntent(),
      },
      child: Actions(
        actions: <Type, Action<Intent>>{
          SaveIntent: CallbackAction<SaveIntent>(onInvoke: (intent) => _onSave()),
          CancelIntent: CallbackAction<CancelIntent>(onInvoke: (intent) => _onCancel()),
        },
        child: Focus(
          autofocus: true,
          child: Scaffold(
            appBar: AppBar(title: const Text('Add Employee Profile (Adaptive Form)'),
            body: SingleChildScrollView(
              padding: const EdgeInsets.all(24.0),
              child: Form(
                key: _formKey,
                child: LayoutBuilder(
                  builder: (context, constraints) {
```

```

final width = constraints.maxWidth;
final int crossAxisCount = width > 1000 ? 3 : (width > 600 ? 2 : 1);

return Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    // Responsive Multi-Column Form Grid
    GridView.count(
      crossAxisCount: crossAxisCount,
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      crossAxisSpacing: 16,
      mainAxisSpacing: 16,
      childAspectRatio: 3.5,
      children: [
        TextFormField(
          decoration: const InputDecoration(labelText: 'Full Name *', border:
            validator: (v) => v == null || v.isEmpty ? 'Required' : null,
          ),
        TextFormField(
          decoration: const InputDecoration(labelText: 'Employee Code *', bor
            validator: (v) => v == null || v.isEmpty ? 'Required' : null,
          ),
        TextFormField(
          decoration: const InputDecoration(labelText: 'Email Address *', bor
            validator: (v) => v != null && v.contains('@') ? null : 'Invalid Em
          ),
        TextFormField(
          decoration: const InputDecoration(labelText: 'Designation', border:
          ),
        TextFormField(
          decoration: const InputDecoration(labelText: 'Department', border:
          ),
        TextFormField(
          decoration: const InputDecoration(labelText: 'Phone Number', border
          ),
      ],
    ),
    const SizedBox(height: 24),

    // Action Buttons
    Row(
      mainAxisAlignment: MainAxisAlignment.end,
      children: [
        OutlinedButton(
          onPressed: _onCancel,
          child: const Text('Cancel (Esc)'),
        ),
        const SizedBox(width: 12),
        ElevatedButton.icon(
          onPressed: _onSave,
          icon: const Icon(Icons.save, size: 18),
          label: const Text('Save Record (Cmd+S)'),
        ),
      ],
    ),
  ],
);
},
),

```

```
    },
  },
),
),
),
);
}
```

Challenge 5: Cross-Platform Image Picker & Uploader (Web Blob vs Mobile Native File)

The Interviewer's Prompt:

"Implement an `ImageUploadService` in Flutter that works seamlessly across Web (memory bytes/Blob) and Mobile (file paths). It must validate that the file size is under 5MB, generate a previewable `Uint8List`, and return an upload-ready payload."

Complete Production Code:

```
import 'dart:typed_data';
import 'package:image_picker/image_picker.dart';

class ProcessedImageResult {
  final String fileName;
  final Uint8List bytes;
  final int fileSizeBytes;
  final String? localFilePath; // Available on Mobile, null on Web

  const ProcessedImageResult({
    required this.fileName,
    required this.bytes,
    required this.fileSizeBytes,
    this.localFilePath,
  });

  double get fileSizeInMb => fileSizeBytes / (1024 * 1024);
}

class CrossPlatformImageUploadService {
  final ImagePicker _picker;
  static const int maxFileSizeBytes = 5 * 1024 * 1024; // 5 MB limit

  CrossPlatformImageUploadService({ImagePicker? picker}) : _picker = picker ?? ImagePicker();

  Future<ProcessedImageResult> pickAndValidateImage({ImageSource source = ImageSource.gallery}) async {
    final XFile? file = await _picker.pickImage(
      source: source,
      maxWidth: 1200, // Downscale to prevent OOM
      maxHeight: 1200,
      imageQuality: 80,
    );

    if (file == null) {
      throw Exception('User cancelled image selection');
    }

    // Read bytes directly via XFile (Compatible with both Web and Mobile!)
    final Uint8List bytes = await file.readAsBytes();
    final int size = bytes.lengthInBytes;

    if (size > maxFileSizeBytes) {
      throw Exception('Image exceeds 5MB limit (${(size / (1024 * 1024)).toStringAsFixed(1)} MB)');
    }

    return ProcessedImageResult(
      fileName: file.name,
      bytes: bytes,
      fileSizeBytes: size,
      localFilePath: file.path.isNotEmpty && !file.path.startsWith('blob:') ? file.path : null,
    );
  }
}
```

Challenge 6: Timesheet Shift Calculator with 24-Hour Auto-Capping & Overtime

The Interviewer's Prompt:

"Write a pure Dart calculation engine that takes attendance check-in and check-out timestamps and computes: gross duration, regular hours (capped at 8.0), 1.0 hr food break deduction if gross $\geq$ 9 hrs, 1.0 hr travel tolerance, and overtime hours (starts after 10 gross hrs). If an unclosed shift exceeds 24 hours, cap regular hours at 8.0 with 0 overtime and flag `isAutoCompleted = true`."

Complete Production Code:

```
class ShiftCalculationResult {
  final Duration grossDuration;
  final double regularHours;
  final double breakHours;
  final double travelToleranceHours;
  final double overtimeHours;
  final bool isCompleted;
  final bool isAutoCompleted;

  const ShiftCalculationResult({
    required this.grossDuration,
    required this.regularHours,
    required this.breakHours,
    required this.travelToleranceHours,
    required this.overtimeHours,
    required this.isCompleted,
    required this.isAutoCompleted,
  });
}

class TimesheetCalculatorEngine {
  static const double standardRegularCap = 8.0;
  static const double foodBreakHours = 1.0;
  static const double travelToleranceHours = 1.0;

  static ShiftCalculationResult calculateShift({
    required DateTime checkIn,
    DateTime? checkOut,
    DateTime? currentTime,
  }) {
    final now = currentTime ?? DateTime.now();

    // 1. Case: Shift currently in progress
    if (checkOut == null) {
      final inProgressDuration = now.difference(checkIn);

      // Shift crossing 24 hours without checkout -> Auto Cap at 8.0 hrs
      if (inProgressDuration >= const Duration(hours: 24)) {
        return const ShiftCalculationResult(
          grossDuration: Duration(hours: 24),
          regularHours: standardRegularCap,
          breakHours: foodBreakHours,
          travelToleranceHours: travelToleranceHours,
          overtimeHours: 0.0,
          isCompleted: true,
          isAutoCompleted: true, // Flagged for Admin Audit
        );
      }

      return ShiftCalculationResult(
        grossDuration: inProgressDuration,
        regularHours: 0.0,
        breakHours: 0.0,
        travelToleranceHours: 0.0,
        overtimeHours: 0.0,
        isCompleted: false,
        isAutoCompleted: false,
      );
    }
  }
}
```

```

    );
}

// 2. Case: Completed normal shift
final gross = checkOut.difference(checkIn);
final grossDecimalHours = gross.inMinutes / 60.0;

double regular = 0.0;
double breakDeduction = 0.0;
double travelDeduction = 0.0;
double overtime = 0.0;

if (grossDecimalHours >= 10.0) {
    // 10+ hours: 8.0 Regular + 1.0 Break + 1.0 Travel + Remainder as Overtime
    regular = standardRegularCap;
    breakDeduction = foodBreakHours;
    travelDeduction = travelToleranceHours;
    overtime = grossDecimalHours - 10.0;
} else if (grossDecimalHours >= 9.0) {
    // 9 to 10 hours: 8.0 Regular + 1.0 Break + partial travel
    regular = standardRegularCap;
    breakDeduction = foodBreakHours;
    travelDeduction = grossDecimalHours - 9.0;
    overtime = 0.0;
} else {
    // Under 9 hours: No overtime
    regular = grossDecimalHours;
    breakDeduction = 0.0;
    travelDeduction = 0.0;
    overtime = 0.0;
}

return ShiftCalculationResult(
    grossDuration: gross,
    regularHours: double.parse(regular.toStringAsFixed(2)),
    breakHours: breakDeduction,
    travelToleranceHours: travelDeduction,
    overtimeHours: double.parse(overtime.toStringAsFixed(2)),
    isCompleted: true,
    isAutoCompleted: false,
);
}
}

```

Challenge 7: Reactive Offline-First Cache-Aside Repository (Stream + In-Memory L1 Cache)

The Interviewer's Prompt:

"Design a repository method `watchEmployees()` that emits cached data from local storage immediately to the UI, performs a background network fetch, updates the cache, and pushes the latest records to the stream without duplicate emits."

Complete Production Code:

```
import 'dart:async';

abstract class EmployeeLocalDataSource {
  Future<List<Employee>> getCacheEmployees();
  Future<void> saveEmployees(List<Employee> employees);
}

abstract class EmployeeRemoteDataSource {
  Future<List<Employee>> fetchRemoteEmployees();
}

class OfflineFirstEmployeeRepository {
  final EmployeeLocalDataSource localData;
  final EmployeeRemoteDataSource remoteData;

  // Broadcast Stream Controller to notify multiple UI listeners
  final _controller = StreamController<List<Employee>>.broadcast();

  // In-Memory L1 Cache to prevent redundant emissions
  List<Employee>? _inMemoryCache;

  OfflineFirstEmployeeRepository({
    required this.localData,
    required this.remoteData,
  });

  Stream<List<Employee>> watchEmployees() {
    // Trigger sync pipeline asynchronously whenever a listener subscribes
    _syncEmployees();
    return _controller.stream;
  }

  Future<void> _syncEmployees() async {
    // Step 1: Emit from in-memory cache if available
    if (_inMemoryCache != null) {
      _controller.add(_inMemoryCache!);
    } else {
      // Step 2: Read from local persistent disk (Hive / SQLite)
      try {
        final cached = await localData.getCacheEmployees();
        if (cached.isNotEmpty) {
          _inMemoryCache = cached;
          _controller.add(cached);
        }
      } catch (_) {
        // Local read error: continue to network
      }
    }

    // Step 3: Fetch fresh data from remote API
    try {
      final remote = await remoteData.fetchRemoteEmployees();

      // Check if remote data differs before emitting
      _inMemoryCache = remote;
      await localData.saveEmployees(remote); // Update disk
      _controller.add(remote); // Emit latest data to UI
    }
  }
}
```

```
    } catch (e) {
        // Network failed: UI already has cached data; emit error to stream listener if empty
        if (_inMemoryCache == null) {
            _controller.addError(Exception('Network error and no local cache available: $e'));
        }
    }
}

void dispose() {
    _controller.close();
}
}
```